

Atelier Flask

Première partie : Flask et son environnement technologique



Flask

Sommaire

Flask vs autres frameworks
Comparaison avec d'autres frameworks :
Django, FastAPI et Streamlit

1

Présentation de Flask
Flask, Werkzeug, WSGI, Jinja

2

3

Architecture et écosystème
Cycle d'une requête et composants

Base de données et ORM
Connexion BDD et SQLAlchemy

4

5

Outils de développement
UV, Ruff & Gunicorn

Mise en pratique - TD
Création de zéro d'une mini appli. Flask

6

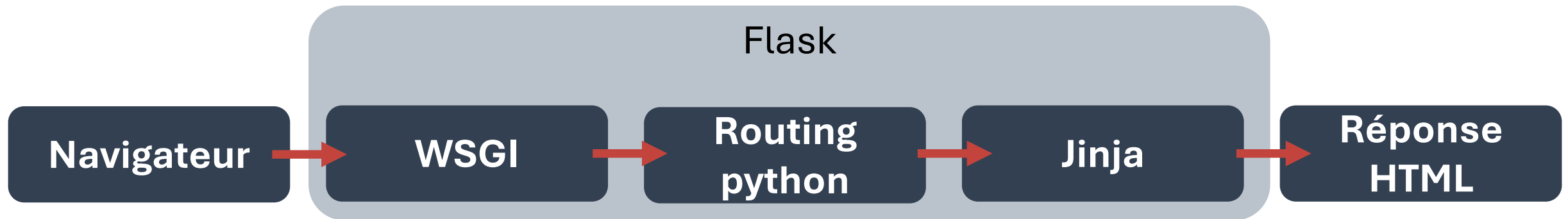
Sommaire

Présentation de Flask

Qu'est-ce que Flask ?

- Présentation de Flask
- Vs autres Frameworks
- Architecture et écosystème

- ➔ Micro-framework web Python
- ➔ Repose principalement sur deux technologies : Werkzeug (WSGI) et Jinja
- ➔ Créé en 2010 par Armin Ronacher et toujours actualisé
- ➔ Lien du site : [FLASK](https://flask.palletsprojects.com/en/2.0.x/)



Présentation de Flask

Werkzeug (« outils » en allemand)

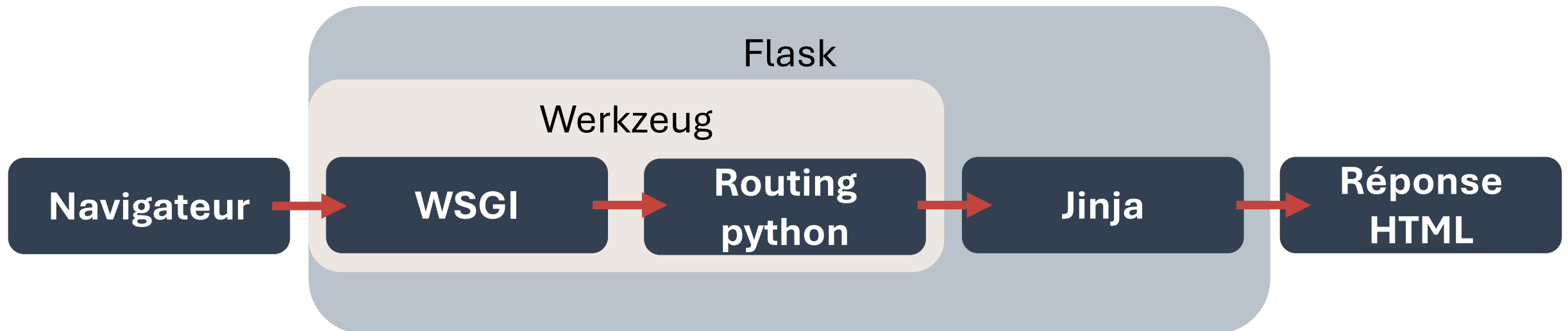
- Présentation de Flask
- Vs autres Frameworks
- Architecture et écosystème

**WSGI : Web Server Gate Interface =
Interface entre serveurs et app. Web pour Python**

➔ Bibliothèque bas-niveau

➔ Fournit toutes les briques pour la création d'une app basée sur WSGI.

➔ Lien du site : [WERKZEUG](https://werkzeug.palletsprojects.com/)



Présentation de Flask

Werkzeug : Qu'est-ce que ça fait ?

- Présentation de Flask
- Vs autres Frameworks
- Architecture et écosystème

WSGI

- « Surcouche » à l'interface web / python WSGI
- Transforme le dictionnaire **CGI** d'informations sur la requête donné par le navigateur en un objet python utilisable facilement.
- Garantit la structure de données et de réponses attendu par WSGI.

CGI : Common Gateway Interface = interface des serveurs HTTP : au lieu d'envoyer un fichier pour une adresse web, on exécute un programme et retourne le contenu généré.

Routing python

- Gère un dictionnaire mappant les URLs à des fonctions python
- Appelle la fonction attribuée à l'URL.
- Encapsule la réponse, gère les arguments d'entrée et la requête.
- Crée un environnement qui possède toutes les informations nécessaires

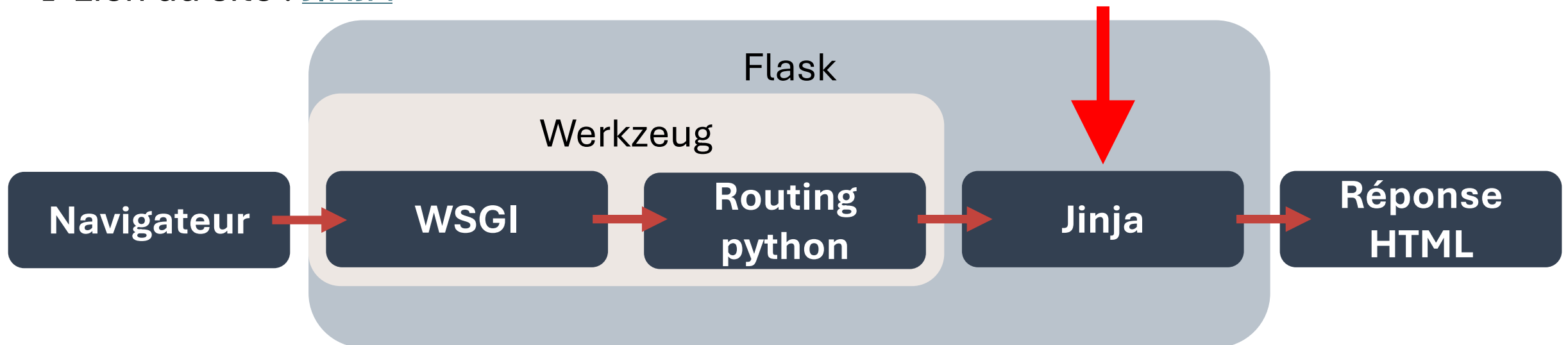
Présentation de Flask

Jinja : Moteur de templates

- Présentation de Flask
- Vs autres Frameworks
- Architecture et écosystème

- ➔ Moteur de template pour Python
- ➔ Utilisé par Flask, mais peut être utilisé seul
- ➔ Protection contre l'injection de code
- ➔ Lien du site : [JINJA](https://jinja.palletsprojects.com/en/3.1.x/)

Moteur de template =
Génère du texte (ici html) dynamiquement à
partir de données Python



Présentation de Flask

Jinja : Moteur de templates

- Présentation de Flask
- Vs autres Frameworks
- Architecture et écosystème

Sans Jinja :

```
name = "Ricco"
html = "<h1>Bonjour " + name + "</h1>"
```

Avec Jinja :

```
>>> render_template(
    "profile.html", name="Ricco"
)
'profile.html' :
<h1>Bonjour {{ name }}</h1>
```

Renvoie :

```
<h1>Bonjour Ricco</h1>
```

Trois syntaxes principales :

Variables :

```
{{ name }}
```

Commentaires :

```
{# Pas affiché #}
```

Structure de contrôles :

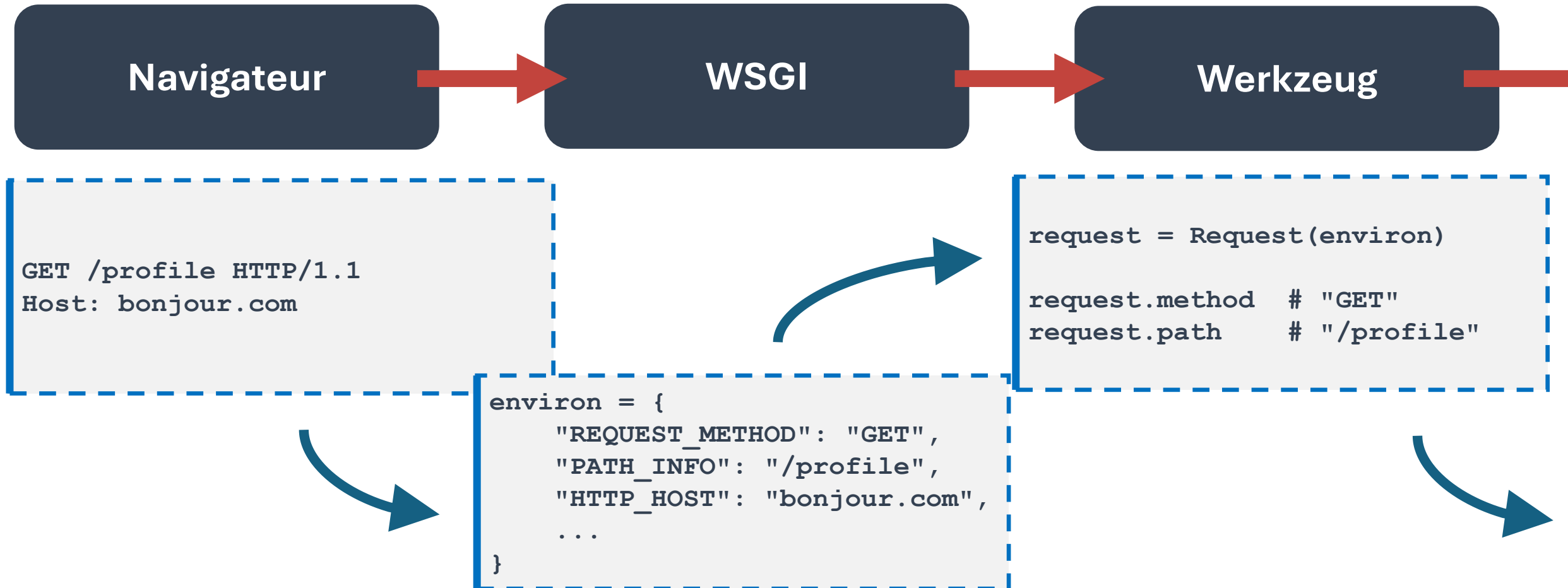
```
{% if user %}
    Bonjour {{ user }}
{% endif %}

{% for item in items %}
    <li>{{ item }}</li>
{% endfor %}
```


Présentation de Flask

Exemple d'une requête

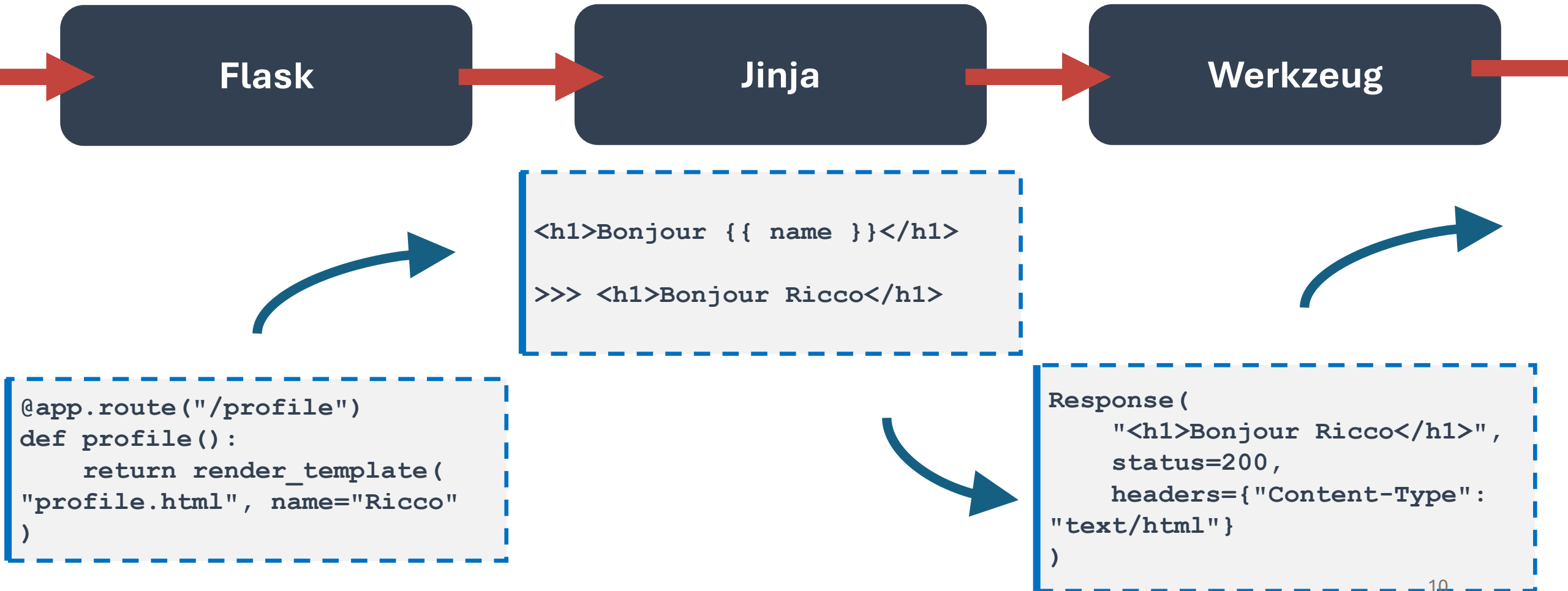
- Présentation de Flask
- Vs autres Frameworks
- Architecture et écosystème



Présentation de Flask

Exemple d'une requête

- Présentation de Flask
- Vs autres Frameworks
- Architecture et écosystème



Présentation de Flask

Exemple d'une requête

- Présentation de Flask
- Vs autres Frameworks
- Architecture et écosystème

WSGI

Réponse HTML

```
[b"<h1>Bonjour Ricco</h1>"]
```

```
HTTP/1.1 200 OK  
Content-Type: text/html  
Content-Length: 24  
  
<h1>Bonjour Ricco</h1>
```

Bonjour Ricco

Présentation de Flask

Initialiser une app

Sans Flask

(besoin d'un serveur WSGI)

```
def app(environ, start_response):
    path = environ.get("PATH_INFO", "/")

    if path == "/":
        response_body = b"Hello World"
        status = "200 OK"
    else:
        response_body = b"Not Found"
        status = "404 Not Found"

    headers = [
        ("Content-Type", "text/plain"),
        ("Content-Length", str(len(response_body)))
    ]

    start_response(status, headers)
    return [response_body]
```

Avec Flask

(en utilisant le serveur WSGI intégré pour le développement)

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello World"

if __name__ == "__main__":
    app.run()
```

App = Flask(__name__):

- ➔ Configure le routing
- ➔ Crée le moteur Jinja
- ➔ Initialisation des structures WSGI
- ➔ Crée les variables de contextes
- ➔ Etc.

- Présentation de Flask
- Vs autres Frameworks
- Architecture et écosystème

Présentation de Flask

Avantages - Inconvénients

- Présentation de Flask
- Vs autres Frameworks
- Architecture et écosystème

Avantages

- **Simplicité** : prise en main facile.
- **Flexibilité** : pas de structure imposée, choix des outils.
- **Léger** : peu de surcouches.
- **Ecosystème riche** : grand nombre d'utilisateurs.

Inconvénients

- **Tout à configurer** : auth, admin, etc.
- **Moins adapté aux gros projets** : scalabilité à désiré.
- **Sécurité incertaine** : dépendante du niveau du/des développeur(s)

Comparaison avec d'autres frameworks

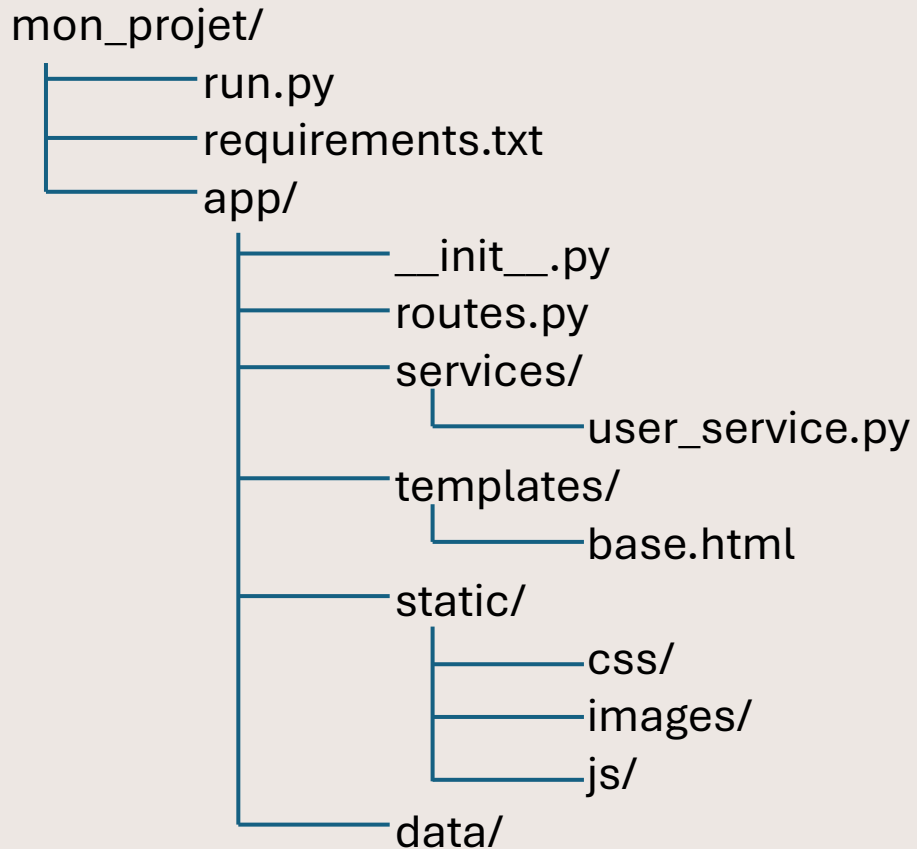
- Vs autres Frameworks
- Architecture et écosystème
- Base de données et ORM

Critère	Flask	FastAPI	Django	Streamlit
Modèle d'exécution	Synchrone (WSGI)	Asynchrone natif (ASGI)	Synchrone (WSGI) + async partiel	Synchrone
Interface serveur	WSGI	ASGI	WSGI (ASGI via Django 3+)	Serveur interne
Typage & validation	Non natif	Typage Python + Pydantic	Forms / Serializers	Non orienté API
ORM intégré	Non	Non	Oui (ORM complet)	Non
Philosophie	Microframework minimaliste	API-first, moderne, typé	Framework full-stack	Data app interactive
Performance brute	Bonne	Très élevée (async + Starlette)	Moyenne à bonne	Non optimisé pour API
Gestion utilisateurs / auth intégrée	Non	Non	Oui (auth complet natif)	Non
Cas d'usage principal	API simple, app légère	API REST haute performance	Applications web complexes	Dashboards data science
Architecture interne	Werkzeug + Jinja	Starlette + Pydantic	Monolithique intégré	Script réactif
Structure imposée	Aucune	Faible	Forte structure MVC	Script linéaire ¹⁴

Architecture et écosystème

Architecture d'un projet Flask

- Architecture et écosystème
- Base de données et ORM
- Outils de développement



app/__init__.py

```
from flask import Flask

def create_app():
    app = Flask(__name__)

    # instantiation des routes :
    from .routes import main_blueprint
    app.register_blueprint(main_blueprint)

    # ici appeler tout ce qui doit être appelé :
    # création de base de données
    # différents services
    # etc.

    return app
```

run.py

```
from app import create_app

app = create_app()
```

Architecture et écosystème

Technologies d'un projet Flask selon leurs couches

- Architecture et écosystème
- Base de données et ORM
- Outils de développement

Navigateur



Côté client
FrontEnd

Routes



Flask

Côté serveur
Api endpoints

Services - BDD

SQLAlchemy



Fonctions et services
Côté serveur
BackEnd

Base de données et ORM

SQLAlchemy : qu'est-ce que c'est ?

- Base de données et ORM
- Outils de développement
- Mise en pratique - TD

SQLAlchemy

**ORM : Object Relational Mapping =
Correspondance objets Python <=> table SQL**

- ➔ Principal ORM utilisé avec Flask
- ➔ Permet de définir des modèles en Python
- ➔ Supporte plusieurs bases
- ➔ Lien du site : [SQLALCHEMY](https://www.sqlalchemy.org/)

Exemple de modèle :

```
class User(db.Model):  
    id = db.Column(db.Integer, primary_key=True)  
    name = db.Column(db.String(100))
```

Base de données et ORM

Créer sa base de données SQLite en utilisant SQLAlchemy

- Base de données et ORM
- Outils de développement
- Mise en pratique - TD

Création de la base de données « users.db »

```
from sqlalchemy import create_engine
from sqlalchemy.orm import declarative_base, sessionmaker

# Définir le moteur SQLite
engine = create_engine("sqlite:///users.db")

# Définir une base déclarative
Base = declarative_base()

# Créer une session pour communiquer avec la BDD
SessionLocal = sessionmaker(bind=engine)
session = SessionLocal()
```

Création le modèle « users »

```
from sqlalchemy import Column, Integer, String, DateTime, ForeignKey,
UniqueConstraint

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True)

    username = Column(String(50),
        nullable=False, unique=True)

    email = Column(String(120),
        nullable=False, unique=True)
```

Base de données et ORM

Communication BDD <=> modèles

- Base de données et ORM
- Outils de développement
- Mise en pratique - TD

Ajouter un individu

```
new_user = User(  
    username="Bob",  
    email=bob@mynameisbob.com)  
session.add(new_user)  
session.commit()
```

Lire les individus

```
users = session.query(User).all()  
# filtrer :  
User_bob = session.query(User).filter_by(  
    username="Bob"  
) .first()
```

Modifier un individu

```
User_bob.email = "bob@gmail.com"  
session.commit()
```

Supprimer un individu

```
session.delete(User_bob)  
session.commit()
```

Base de données et ORM

Contraintes et bonnes pratiques

- Base de données et ORM
- Outils de développement
- Mise en pratique - TD

Contrainte dans la création de modèles

```
class User(Base):  
    __tablename__ = "users"  
  
    id = Column(Integer, primary_key=True)  
  
    username = Column(String(50), nullable=False, unique=True)  
  
    email = Column(String(120), nullable=False, unique=True)  
  
    __table_args__ = (  
        UniqueConstraint('email', name='uq_user_email'), )
```

Liste de contraintes possibles :
Unique, Check, Index, ForeignKey,
Enum, Composites, Validation ORM...

Bonnes pratiques

- **Séparer les modèles, les services, la création de la base de données**
- **Une seule instance de « engine »**
- **Une session par requête: (contextmanager)**
- **Validation à renforcer**
- **Alembic pour les migrations**

Outils de développement

Premier outil - UV

- Outils de développement
- Mise en pratique - TD

- ➔ Gestionnaire de dépendances et d'environnement Python ultra-rapide écrit en Rust.
- ➔ Remplace pip et venv, 10-100x plus rapide
- ➔ Lien du site : [UV](https://docs.astral.sh/uv/)

```
Ajouter un package avec uv :  
> uv add __package_name__
```

```
Enlever un package avec uv :  
> uv remove __package_name__
```

```
Démarrer un nouveau projet uv :  
> uv init  
> uv venv  
> .venv\Scripts\activate
```

```
Récupérer un projet uv :  
> pip install uv  
> uv venv  
> .venv\Scripts\activate  
> uv sync
```

Outils de développement

Deuxième outil - Ruff

- ➔ Formatter et linter python extrêmement rapide
- ➔ Remplace Flake8 /Pylint, 10-100x plus rapide
- ➔ Garantit une base de code propre et uniforme
- ➔ Très utile en projet collaboratif
- ➔ Lien du site : [RUFF](https://docs.astral.sh/ruff/)

Vérifier le code :

```
> ruff check .
```

Et le corriger :

```
> ruff check . --fix
```

Formatter le code :

```
> Ruff format .
```

- Outils de développement
- Mise en pratique - TD

```
import pandas as pd
import os,sys

def create_list( ):
    mylist=[1,2,3,4,5]
return mylist
```



```
import os
import sys

import pandas as pd

def create_list():
    mylist = [1, 2, 3, 4, 5]
return mylist
```

Outils de développement

Troisième outil - Gunicorn

- Outils de développement
- Mise en pratique - TD

- ➔ Serveur WSGI utilisé pour déployer des applications Python.
- ➔ Mieux que le serveur Flask intégré car : multi-process, stable pour la mise en production, compatible WSGI
- ➔ Lien du site : [GUNICORN](https://gunicorn.org/)

```
Lancer un serveur flask avec gunicorn :  
> uv add gunicorn  
ou alors  
> pip install gunicorn  
Puis :  
> gunicorn app:app  
Avec plusieurs workers :  
> gunicorn app:app --workers 4
```

Mise en pratique - TD

Première partie du TD : découverte de Flask

● Mise en pratique - TD

Objectif : Créer sa première application web Flask.

- ➔ Initialiser le projet avec UV.
- ➔ Créer une app Flask minimale.
- ➔ Créer un template HTML avec Jinja.

```
▼ 01 - Hello world
  ▼ templates
    <> index.html
  📄 app.py
  ⚙️ pyproject.toml
  ⓘ README.md
```


Merci de nous avoir écouté

Des questions ?

Au travail !

Atelier Flask

Deuxième partie : RagMe



RAGme

Sommaire

Architecture de l'app
Front et Back Ends, BDD, routing

1

Présentation de l'app
Présentation du projet

2

3

Deuxième partie du TD
Implémentation guidée sur un projet à trous

Présentation de l'app RAGme

Ragme : présentation du projet

- Présentation de l'app
- Architecture de l'app
- Deuxième partie du TD

- ➔ Chatbot basé sur la **RAG**
- ➔ Basé sur Mistral AI
- ➔ Gestion des documents, des conversations
- ➔ Pipeline RAG
- ➔ Interface dynamique

**RAG : Retrieval-Augmented Generation =
Technique d'optimisation d'IAg pour exploiter des
ressources supplémentaires sans réentraînement**

Architecture de l'app

Architecture du projet

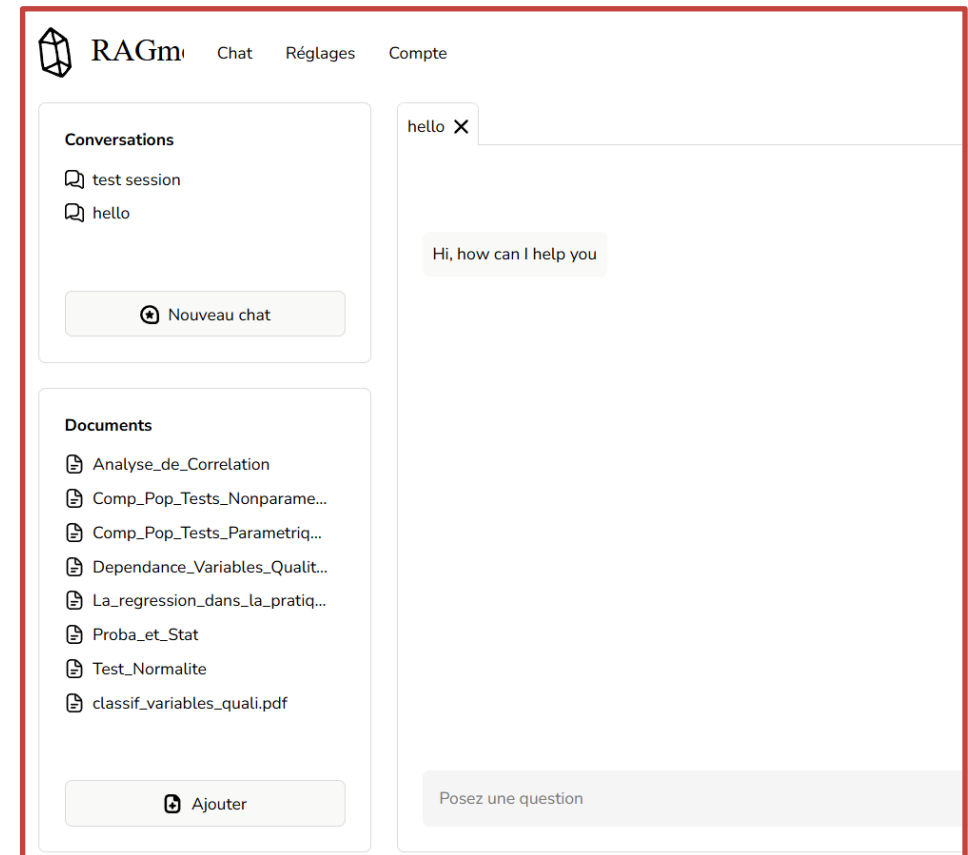


RAGme

- Architecture de l'app
- Deuxième partie du TD

02 - RAGme/

run.py	← Point d'entrée
app/	
__init__.py	← Factory create_app(), init services
routes.py	← Blueprint "main" : route page (/)
ajax.py	← Blueprint "ajax" : API interne (/ajax/*)
models/	← ORM SQLAlchemy
schémas/	← Schémas Pydantic (détachés de la session)
database/	← Gestion des connexions
services/	← Logique métier
rag/	← Coeur du RAG
parser/	← Extraction de contenu
templates/	← Templates Jinja2
static/	← CSS, JS, images

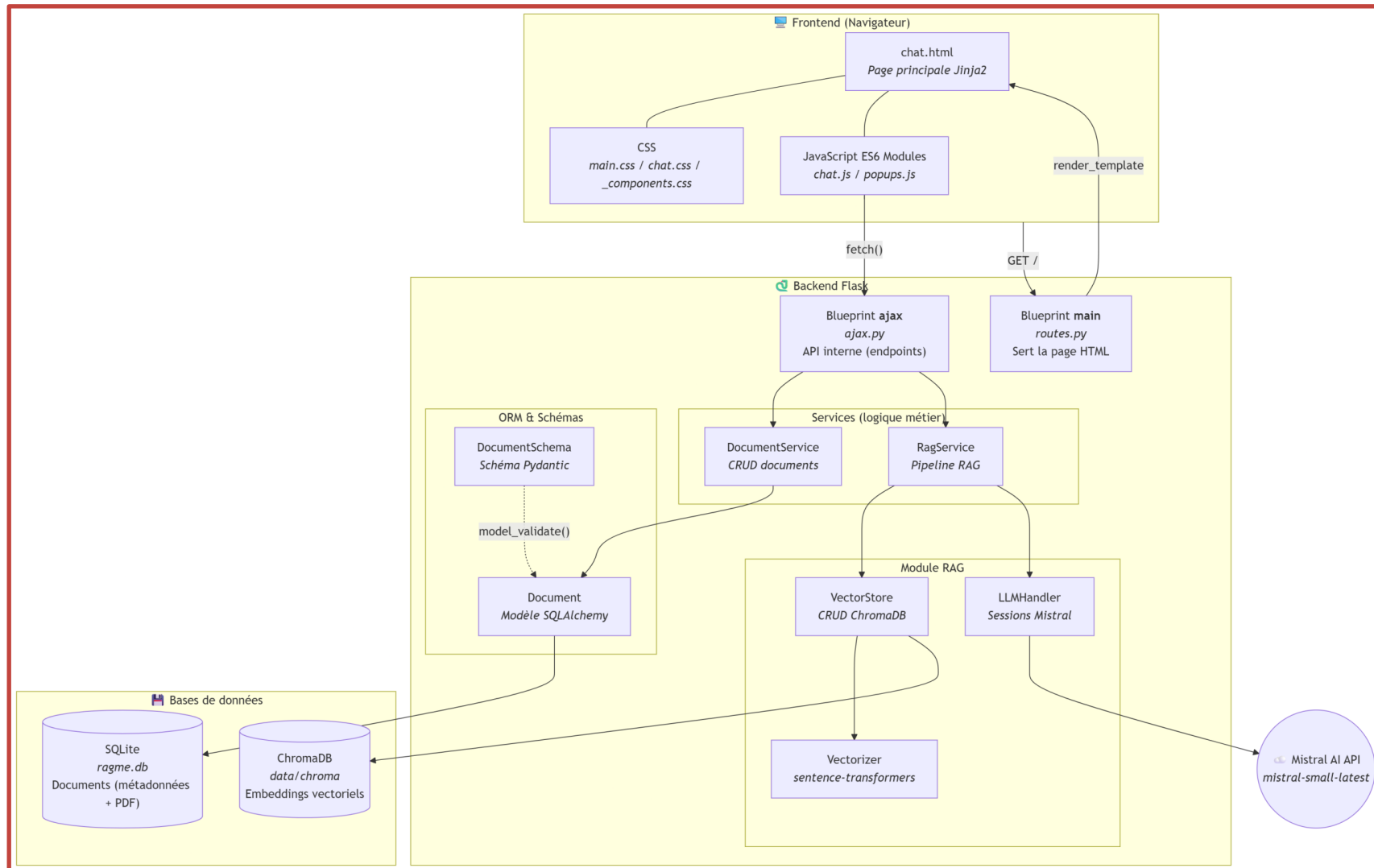


Architecture de l'app



- Architecture de l'app
- Deuxième partie du TD

Front et Back Ends, BDD, routing

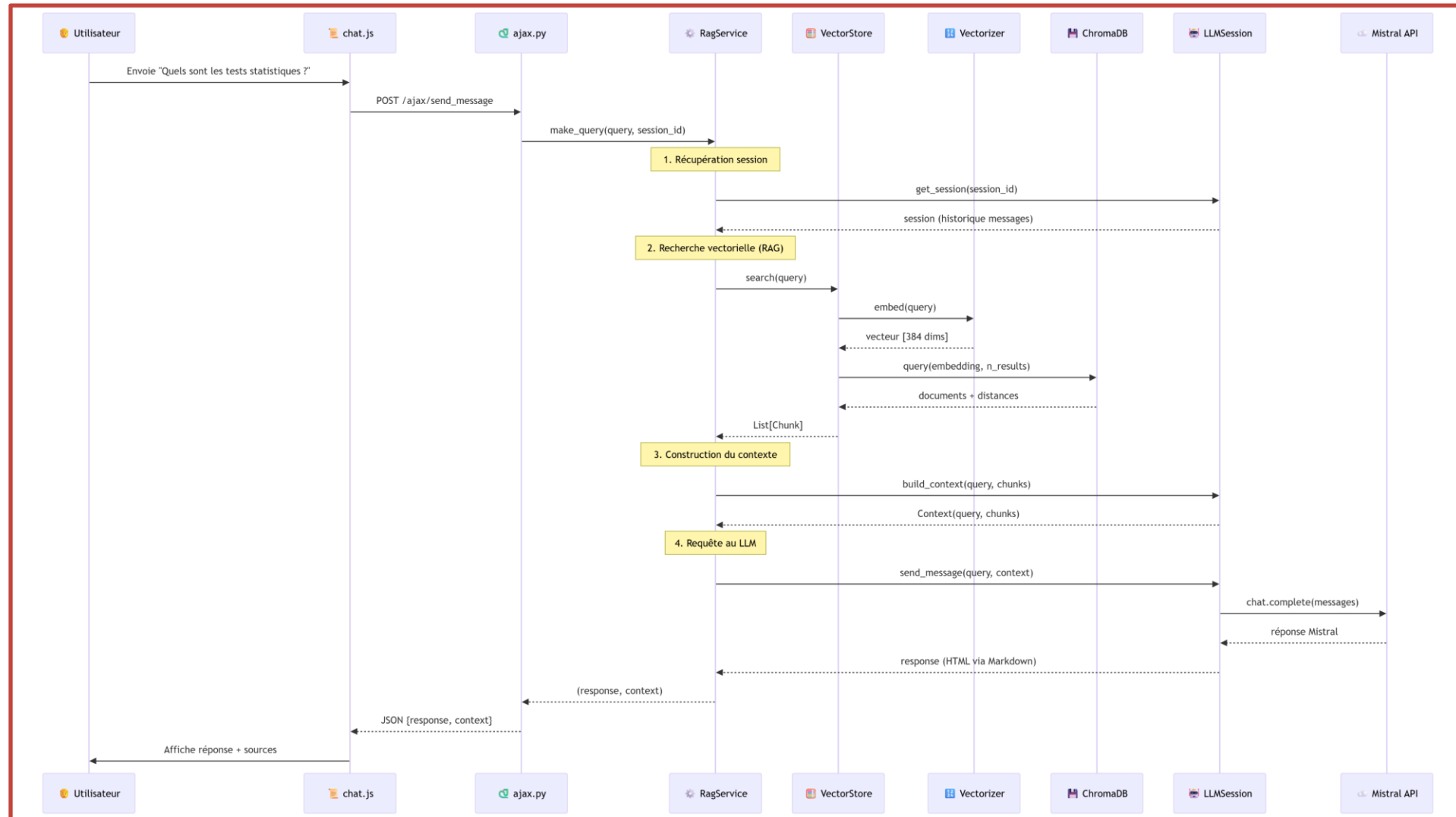


Architecture de l'app



- Architecture de l'app
- Deuxième partie du TD

Front et Back Ends, BDD, routing



Deuxième partie du TD

Implémentation guidée sur un projet à trous

● Deuxième partie du TD

Objectif : Compléter le projet à trous (40 questions/étapes)

→ Créer des routes

→ Configurer la base de données

→ Configurer les services pour la récupération des données

→ Communication API interne (AJAX) : lecture et écriture

→ Téléverser des fichiers



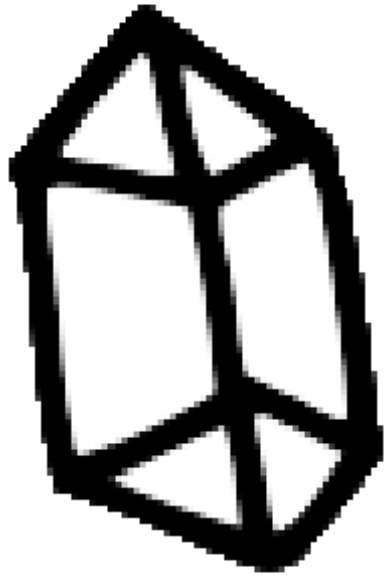
RAGme

Merci de nous avoir écouté

Des questions ?

Au travail !

Annexes

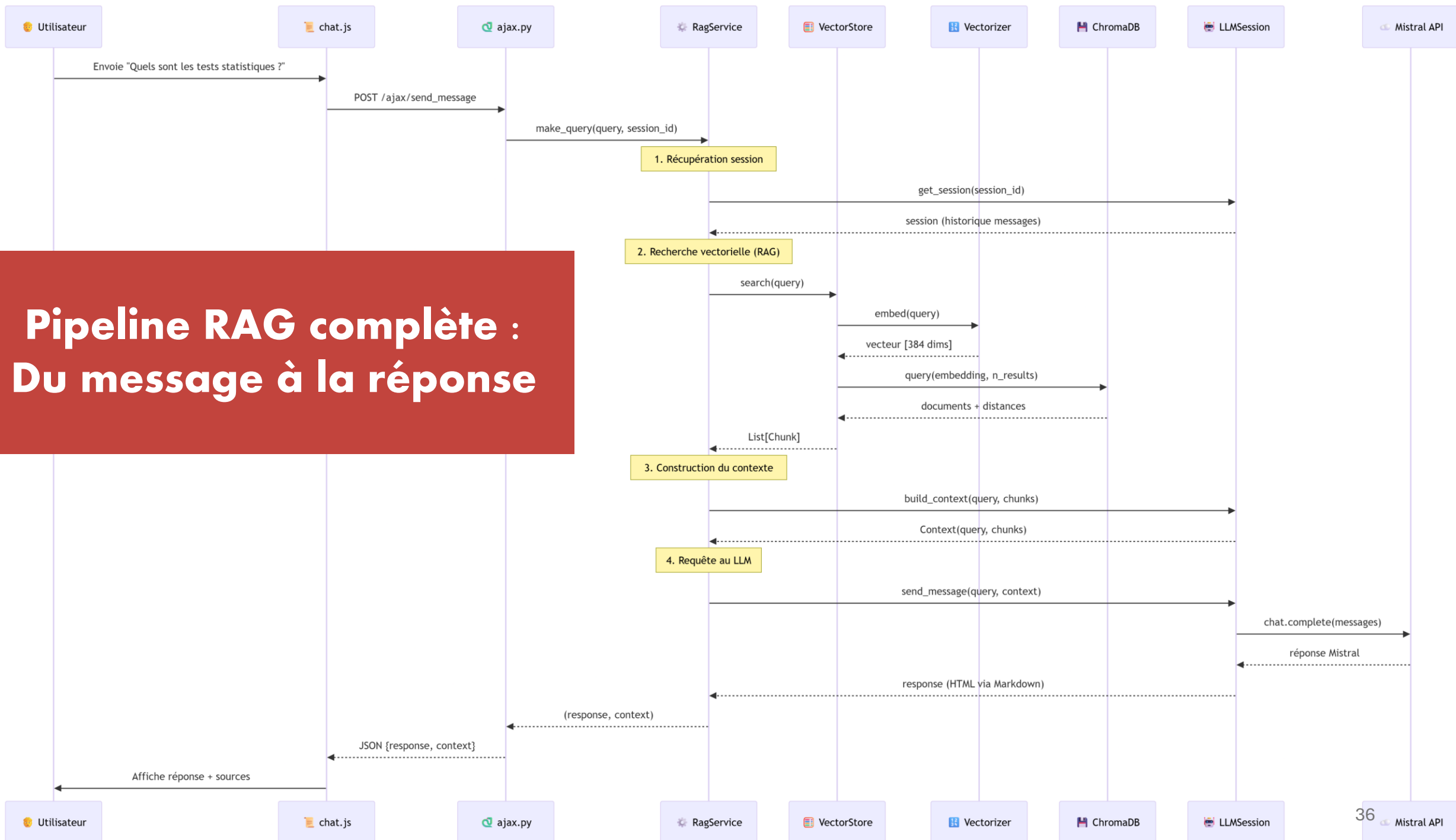


RAGme

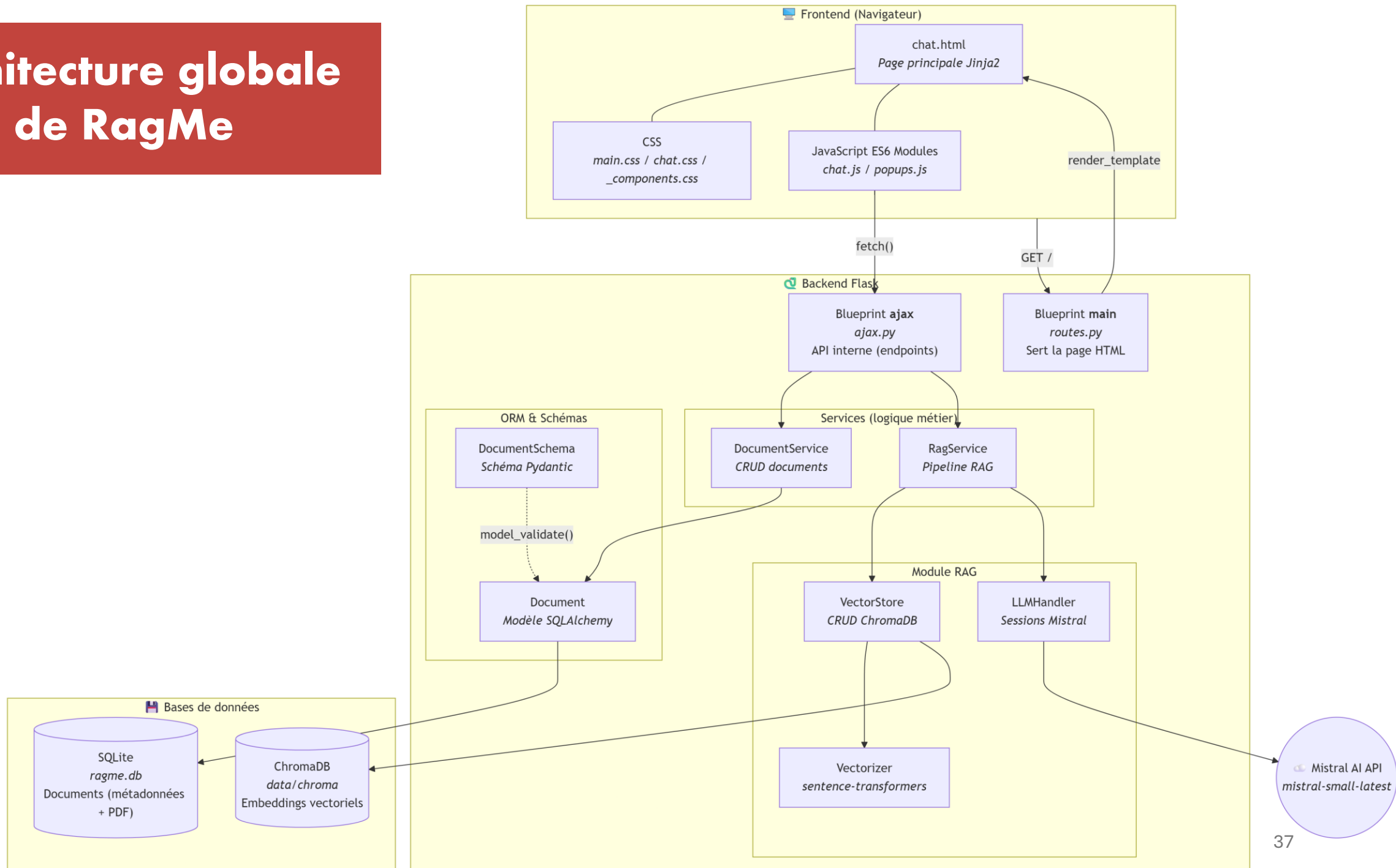
Diagrammes UML RagMe

En grand

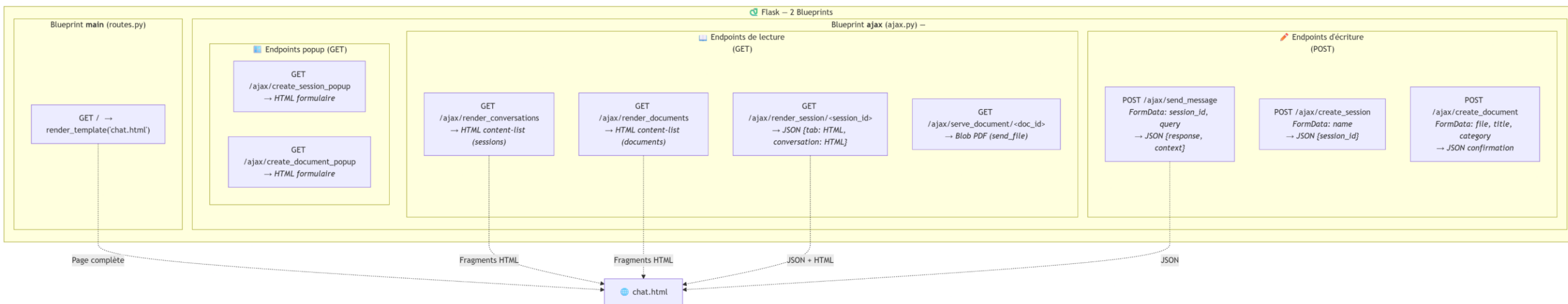
Pipeline RAG complète : Du message à la réponse



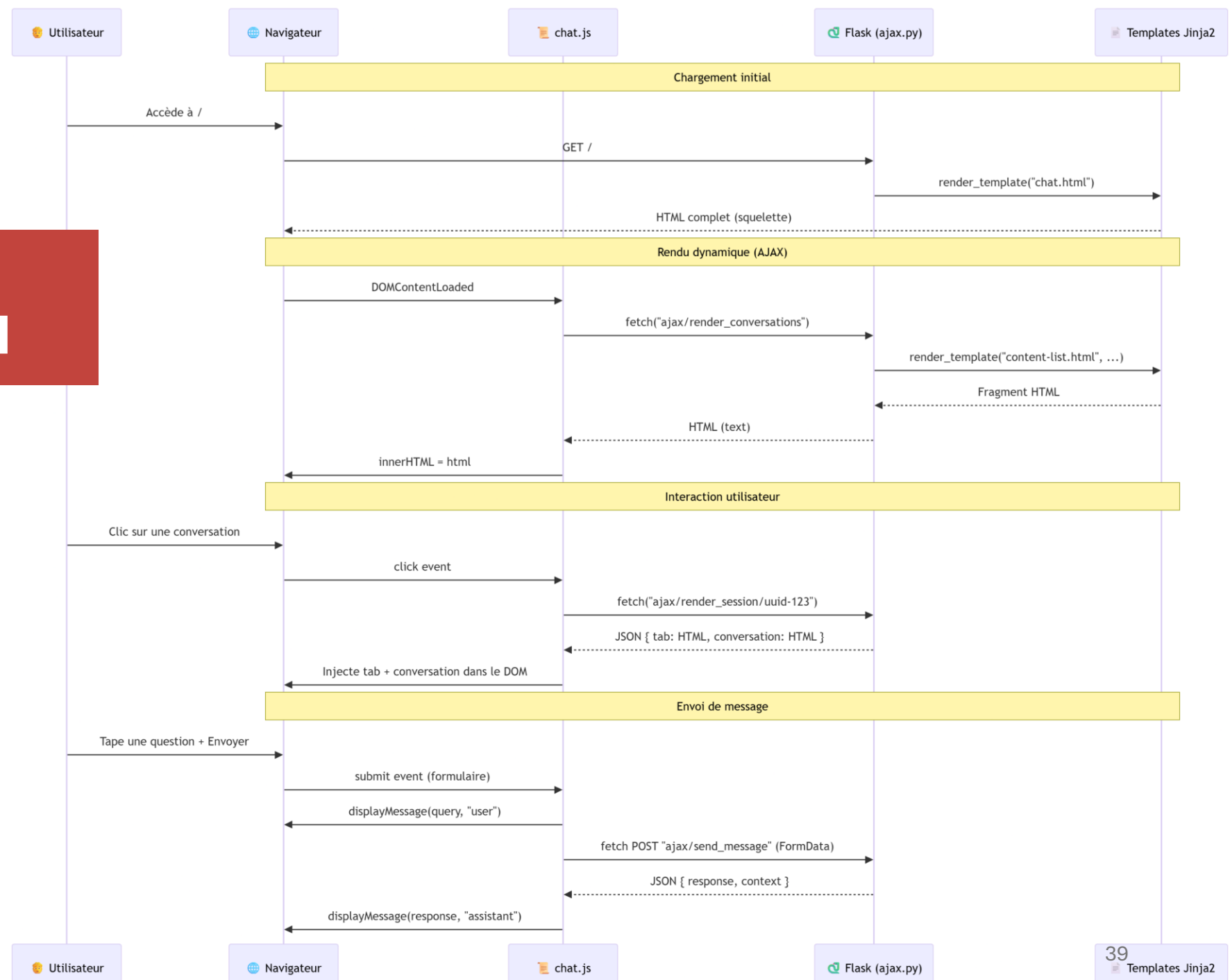
Architecture globale de RagMe



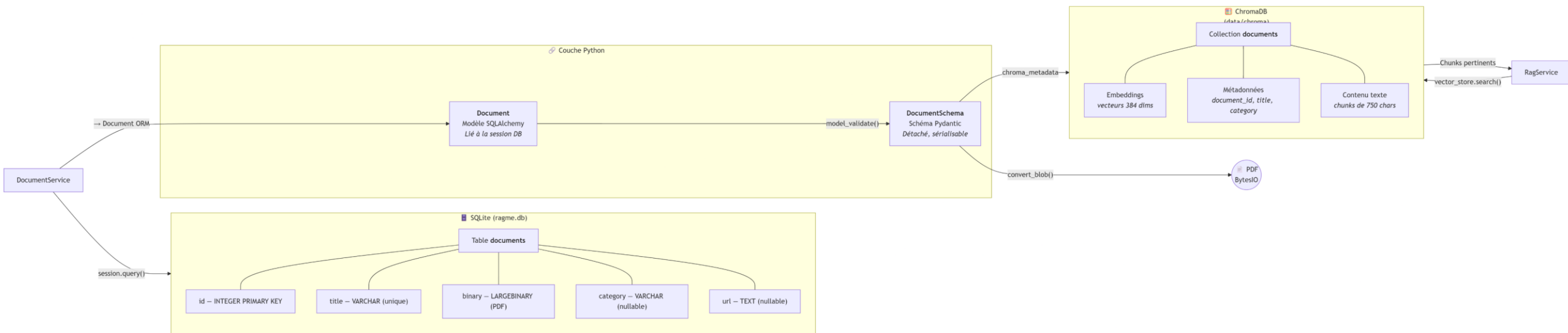
Toutes les routes de l'application



Communication FrontEnd BackEnd



Base de données SQL + vectorielle



FIN